

IoT system design methodology explained using a smart home light control example.

✅ Step 1: Purpose & Requirements (What & Why?)

This step answers: *Why are we building this system? What should it do?*

In your example:

- **Purpose:** Control home lights remotely using a web app
- **Behavior:**
 - Auto mode → light turns ON/OFF automatically based on brightness
 - Manual mode → user controls light remotely
- **Other requirements:**
 - Remote monitoring
 - Local data processing
 - Secure login (authentication)
 - App runs locally but accessible remotely

Simple idea:

Define the goal + features + constraints before building anything.

✅ Step 2: Process Specification (How it works?)

This step answers: *How does the system behave step-by-step?*

Two modes:

◆ Auto Mode:

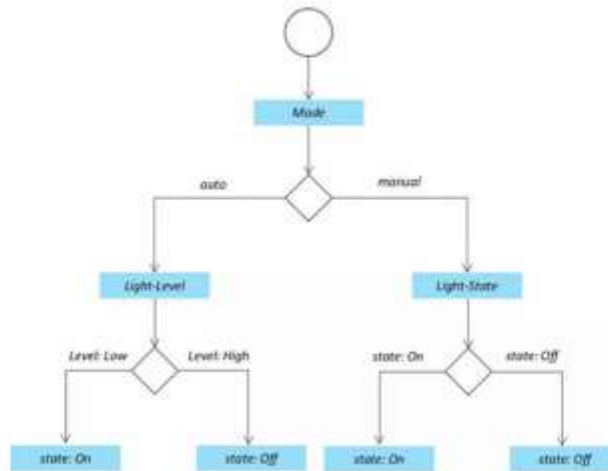
- Read light sensor
- If dark → turn light ON
- If bright → turn light OFF

◆ Manual Mode:

- Check user command

- If ON → turn light ON
- If OFF → turn light OFF

Step:2 - Process Specification



✓ Step 3: Domain Model (Main components)

This step identifies all **important elements in the system**

Key elements:

- **Physical Entity:** Real-world things
 - Room, Light
- **Virtual Entity:** Digital version Digital version of physical objects
 - Room data, Light status
- **Device:** Hardware
 - Raspberry Pi / ESP32 + sensor + relay
- **Resource:** Software components
 - OS, database
- **Service:** Functions that control system
 - Mode control
 - Light control
 - Auto controller

[illegible]

This step defines: *What data are we storing?*

Example:

- Room $\rightarrow$ light level (brightness)
- Light $\rightarrow$ ON/OFF state

Simple idea:

Define **data + attributes** (like variables in programming).

Step 4: Information Model Specification



✓ Step 5: Service Specification (System functions)

This step defines: *What services/APIs will the system provide?*

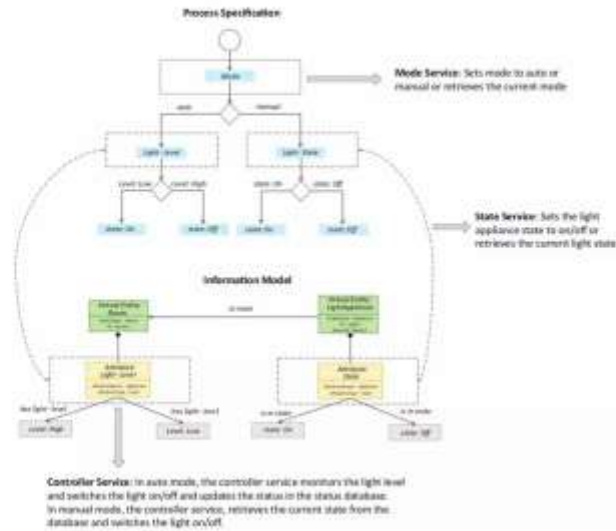
Services:

- **Mode Service**
 - Set auto/manual (PUT)
 - Get current mode (GET)
- **State Service**
 - Turn light ON/OFF (PUT)
 - Get light state (GET)
- **Controller Service**
 - Auto mode → control using sensor
 - Manual mode → follow user command

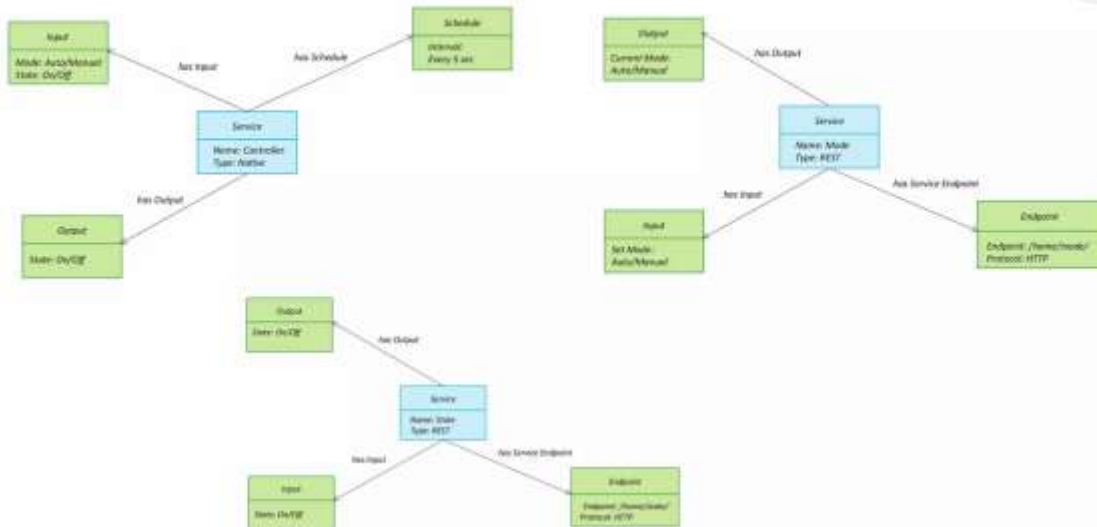
Simple idea:

Define **APIs / functions** that control the system.

Step 5: Service Specifications



Step 5: Service Specifications



✓ Step 6: IoT Level Specification

This step decides system complexity level

In your example:

- Level 1 IoT system

- Device + local processing + simple remote access

Simple idea:

Choose how **simple or complex** your IoT system is.

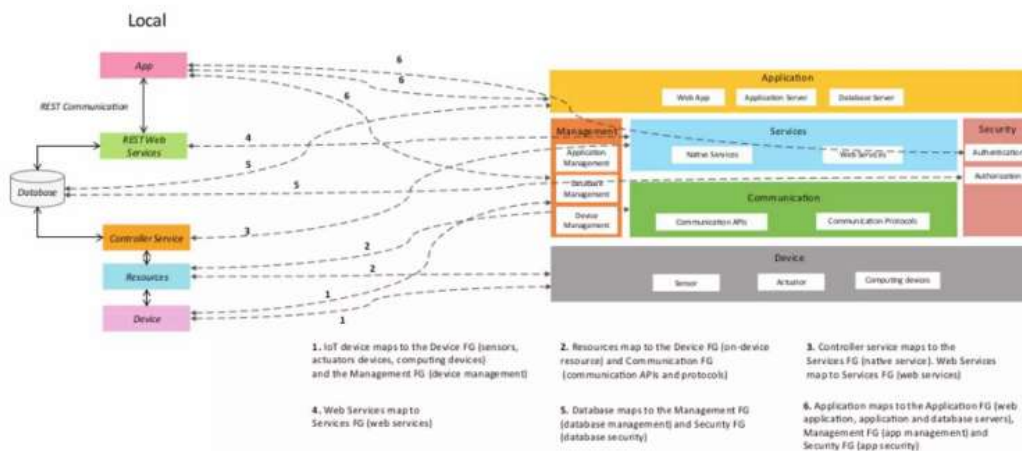
✓ Step 7: Functional View (System groups)

This step groups system into **functional parts**

Groups:

- **Device:** sensors, actuators
- **Communication:** WiFi, HTTP, TCP/IP
- **Services:** APIs + controller
- **Management:** system control, database
- **Security:** login, authentication
- **Application:** web app

Step 7: Functional View Specification



as services for device monitoring, device control services, data publishing services and services for device discovery. In the home automation example, there are two REST services (mode and state service) and one native service (controller service).

- **Management** : The management FG includes all functionalities that are needed to configure and manage the IoT system.
- **Security** : The security FG includes security mechanisms for the IoT system such as authentication, authorization, data security, etc.
- **Application** : The application FG includes applications that provide an interface to the users to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and the processed data.

Figure 5.10 shows an example of mapping deployment level to functional groups for home automation IoT system.

IoT device maps to the Device FG (sensors, actuators devices, computing devices) and the Management FG (device management). Resources map to the Device FG (on-device resource) and Communication FG (communication APIs and protocols). Controller service maps to the Services FG (native service). Web Services map to Services FG . Database maps to the Management FG (database management) and Security FG (database security). Application maps to the Application FG (web application, application and database servers), Management FG (app management) and Security FG (app security).

✓ Step 8: Operational View (Implementation details)

This step defines: *How exactly will we build it?*

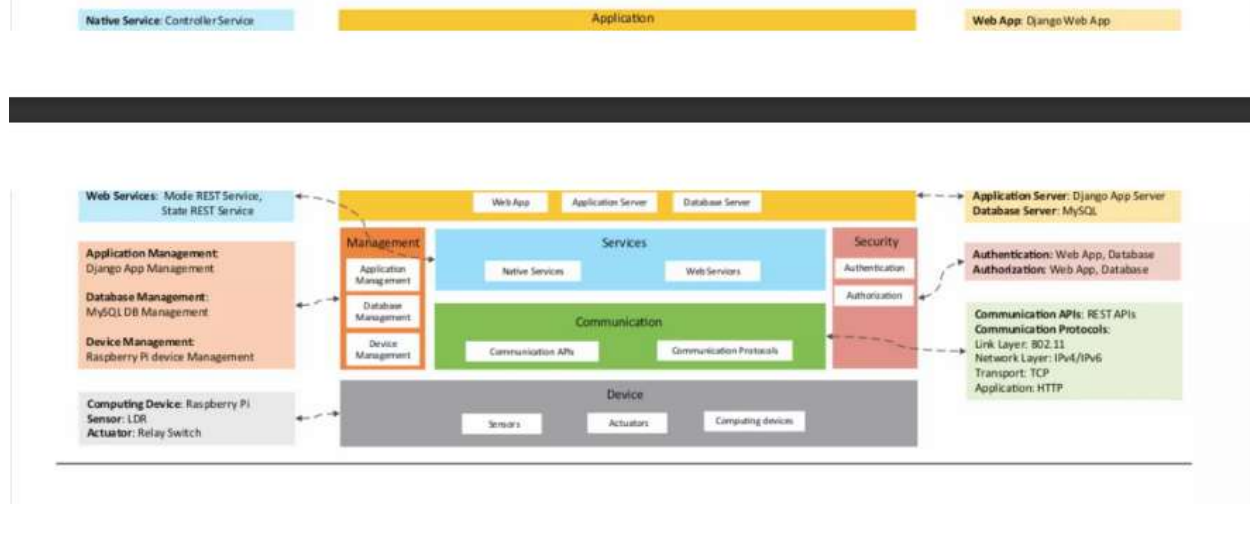
Example choices:

- Device → Raspberry Pi
- Sensor → LDR
- Actuator → Relay
- Protocols → WiFi, HTTP, TCP
- Backend → Django
- Database → MySQL

Simple idea:

Choose **technologies, tools, and platforms**.

Step 8: Operational View Specification



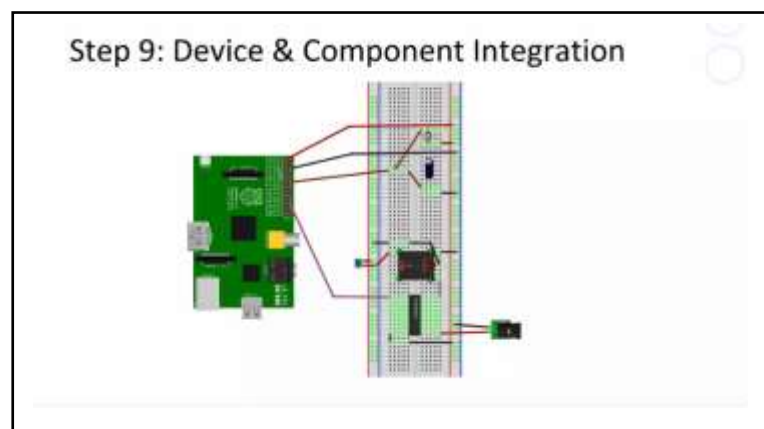
✓ Step 9: Device & Component Integration

This step is actual **hardware setup**

- Connect LDR sensor to board
- Connect relay to control light
- Setup Raspberry Pi / ESP32

Simple idea:

Physically connect everything.



✓ Step 10: Application Development (Final product)

This step builds the **user interface**

Features:



- Button for Auto/Manual mode
- Button for Light ON/OFF
- Show current light status

Simple idea:

Build the **web/mobile app** for users.

IoT Weather Monitoring System (Step-by-Step Simple Flow)

✓ Step 1: Purpose & Requirements (What & Why?)

👉 Purpose:

Build a system that collects weather data (temperature, humidity, pressure, light) from different locations and sends it to the cloud.

👉 Requirements:

- Measure:
 - Temperature
 - Humidity
 - Pressure
 - Light intensity
- Send data to cloud
- Store and analyze data

Purpose :

To collect data on environmental conditions; temperature, pressure, humidity and light in an area using multiple end nodes. The end nodes send the data to the cloud where the data is aggregated and analyzed.

Weather Monitoring

- View data remotely (web/app)
- Multiple sensor nodes
- Secure data transfer

👉 **Simple idea:**

👉 “We want to monitor weather remotely using sensors + cloud.”

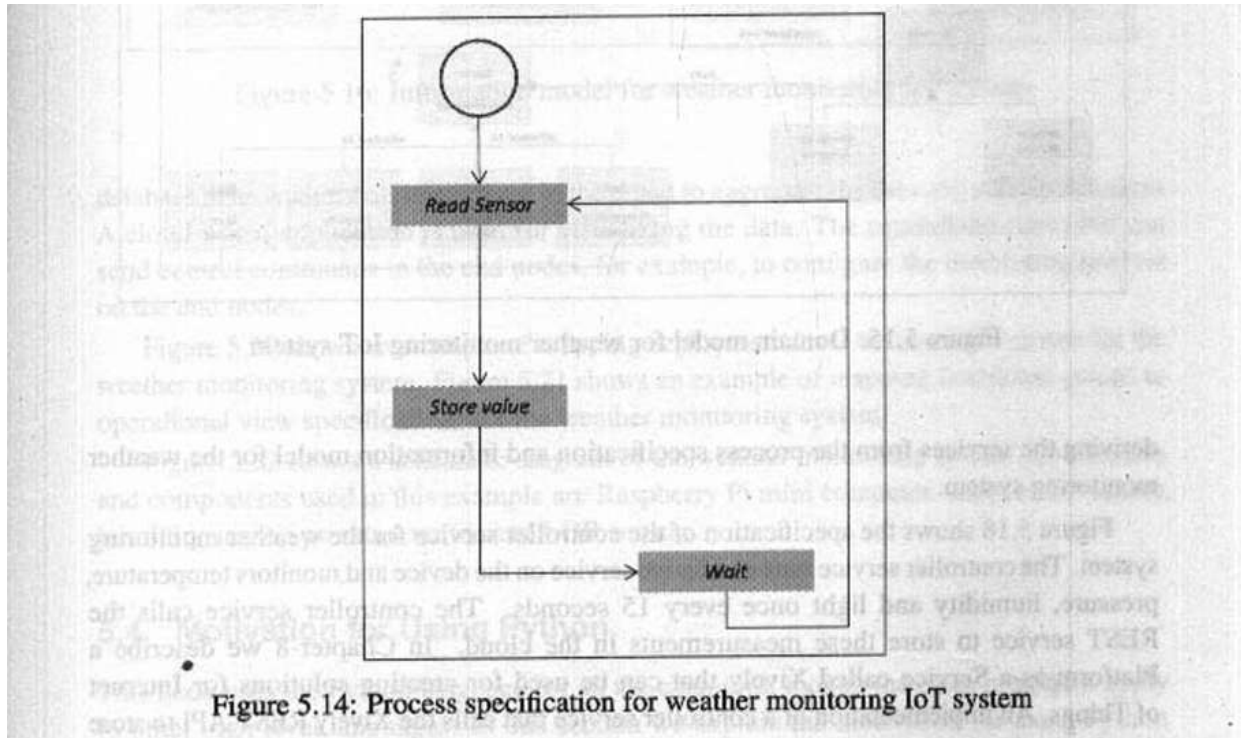
✅ **Step 2: Process Specification (How it works?)**

👉 **System flow:**

1. Start system
2. Read sensors (temperature, humidity, pressure, light)
3. Wait for fixed time (e.g., 15 seconds)
4. Send data to cloud
5. Store data
6. Repeat continuously

👉 **Simple idea:**

👉 “Read → Send → Store → Repeat”



✅ Step 3: Domain Model (Main Components)

👉 Identify all entities:

◆ Physical Entity:

- Environment (weather conditions)

◆ Virtual Entity:

- Digital version of environment (data in cloud)

◆ Devices:

- Temperature sensor
- Humidity sensor
- Pressure sensor
- Light sensor
- Raspberry Pi / ESP32

◆ Resources:

- On-device software
- Cloud database
- ◆ **Services:**
 - Controller service (reads sensors & sends data)

👉 **Simple idea:**

👉 “List all hardware + software + system parts”

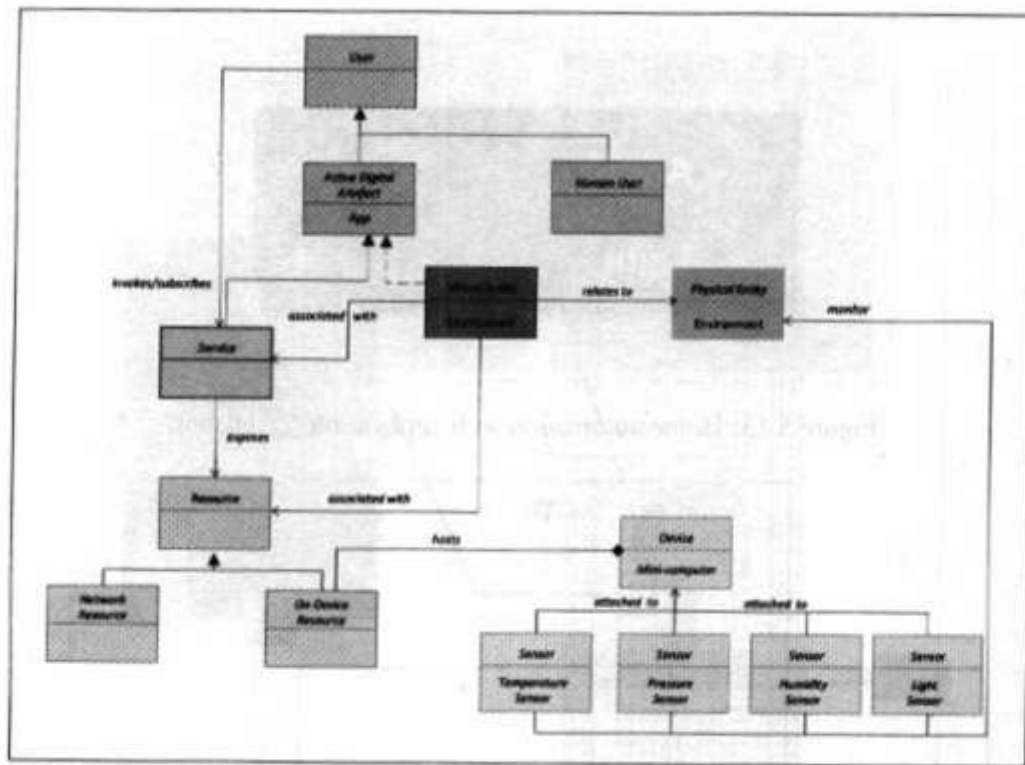


Figure 5.15: Domain model for weather monitoring IoT system

✅ Step 4: Information Model (Data Structure)

👉 Define what data we store:

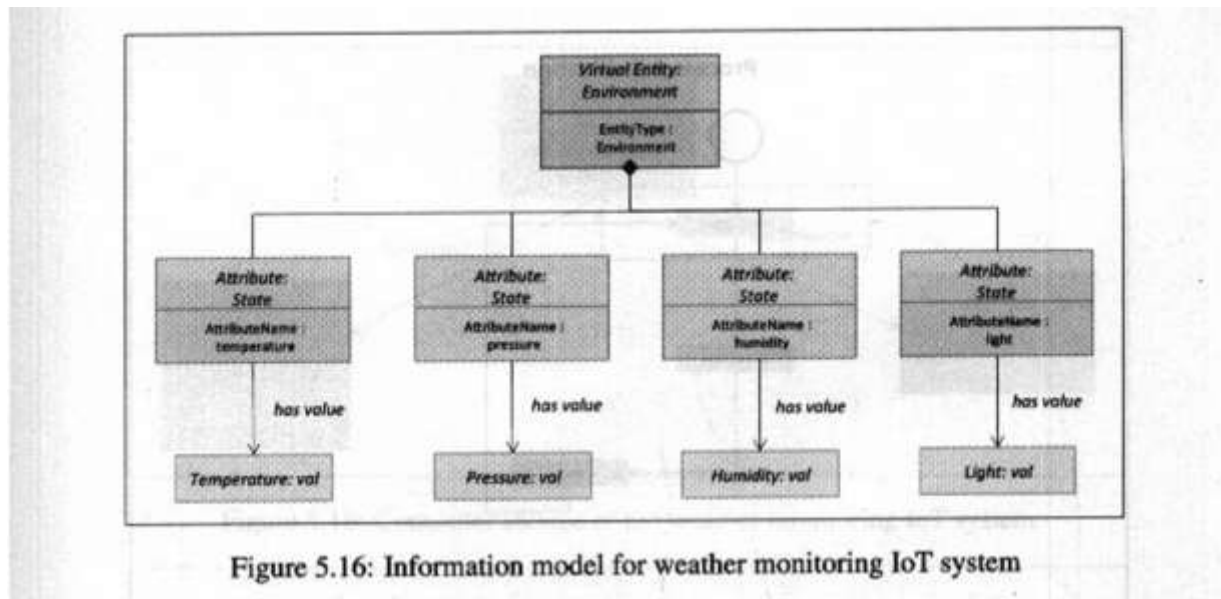
- Temperature
- Humidity
- Pressure
- Light

👉 Each reading is linked to:

- Time
- Device ID (optional but important)

👉 **Simple idea:**

👉 “What data? → Weather values”



✅ Step 5: Service Specification (Functions/APIs)

👉 Controller Service

- Reads sensors every 15 sec
- Sends data to cloud using REST API

👉 Example:

- POST/PUT → send data to cloud
- GET → retrieve data

Alert service (if temperature too high)

- Data analytics service

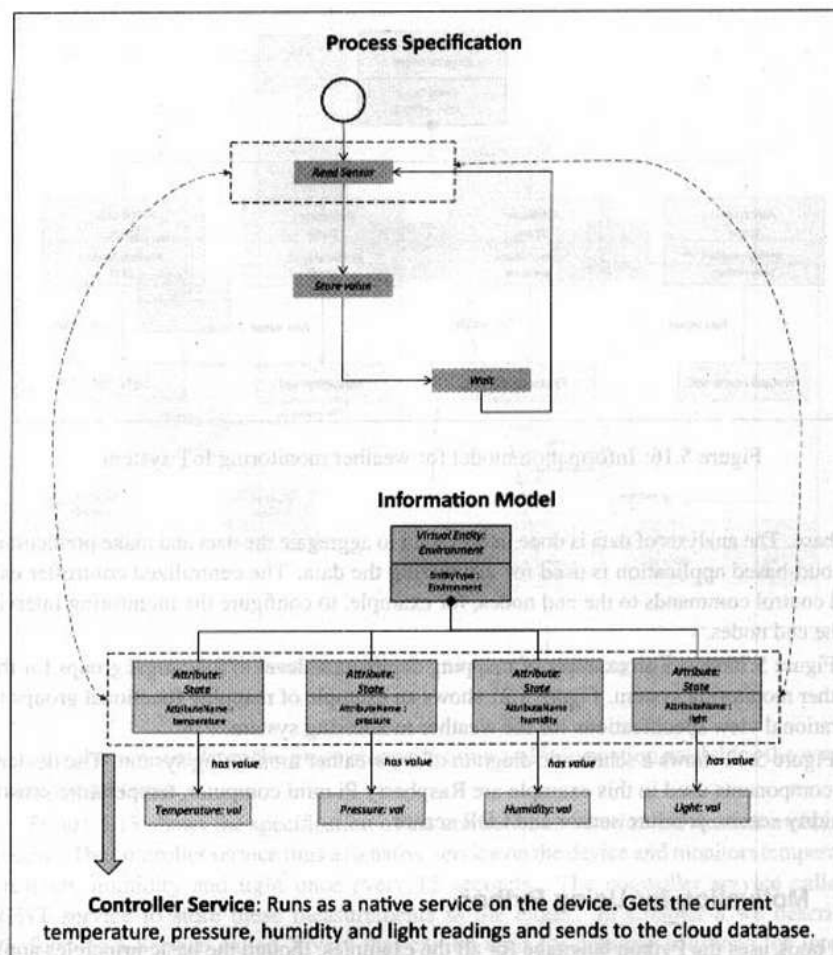


Figure 5.17: Deriving services from process specification and information model for weather monitoring IoT system

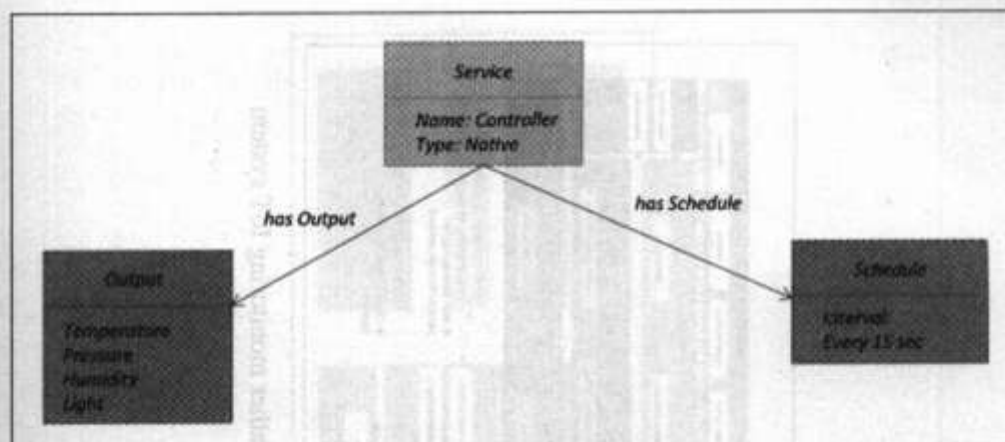
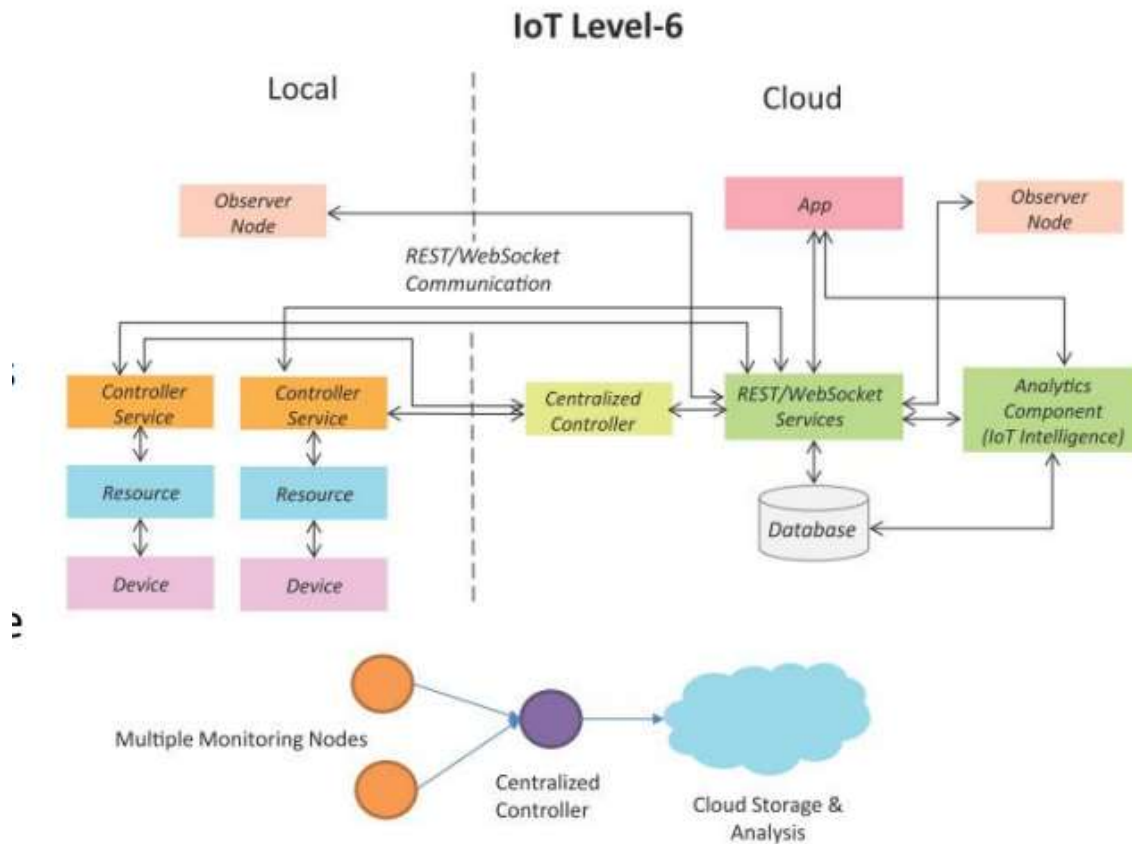


Figure 5.18: Controller service of the weather monitoring IoT system

✓ Step 6: IoT Level Specification

✓ This system is:

- Higher than Level 6 (Cloud-based IoT)



✓ Step 7: Functional View

Functional Groups:

- **Device:** Sensors + Raspberry Pi
- **Communication:** WiFi, Internet
- **Services:** Controller service + Data collection service
- **Management:** Configure nodes (Device configuration, Data management)
- **Security:** Data protection + Authentication
- **Application:** Cloud dashboard

👉 Simple idea:

📌 “Divide system into functional parts”

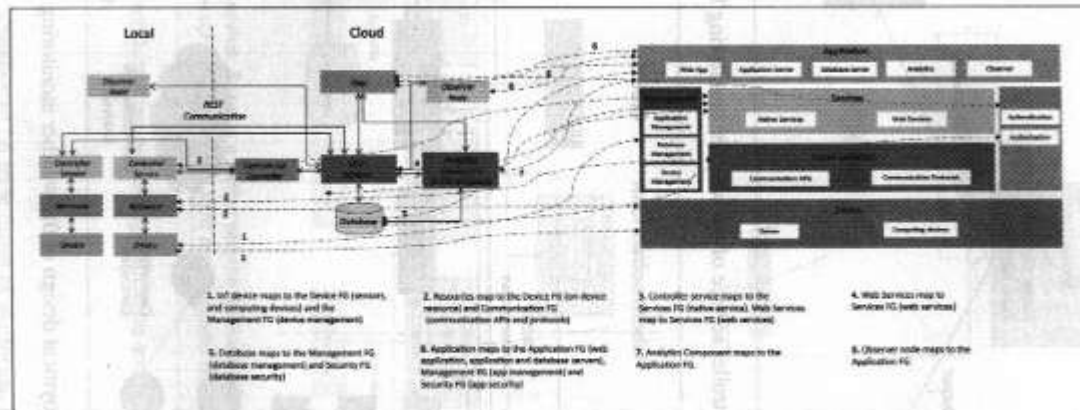


Figure 5.20: Mapping deployment level to functional groups for the weather monitoring IoT system

✅ Step 8: Operational View (Real Tech Used)

Example implementation:

- Device → Raspberry Pi / ESP32
- Sensors → DHT11 (temp+humidity), BMP180 (pressure), LDR
- Protocol → HTTP or MQTT
- Cloud → Xively / Firebase / AWS IoT
- Database → MySQL / Cloud DB
- App → Web dashboard

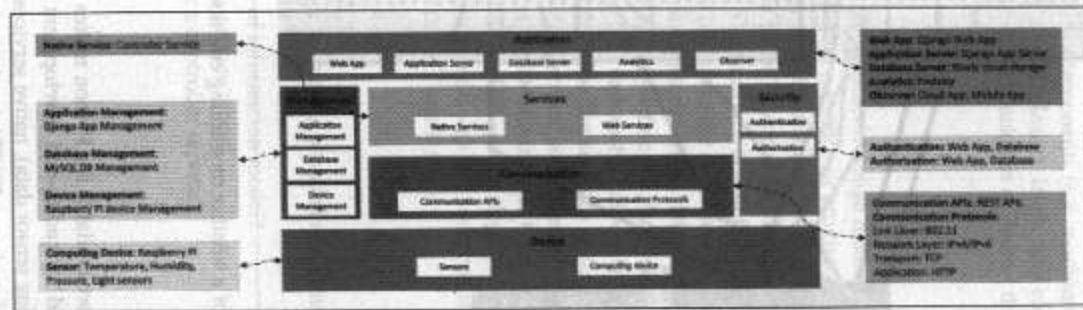


Figure 5.21: Mapping functional groups to operational view for the weather monitoring IoT system

✓ Step 9: Device & Component Integration

Connect:

- Sensors → Raspberry Pi
- Raspberry Pi → Internet

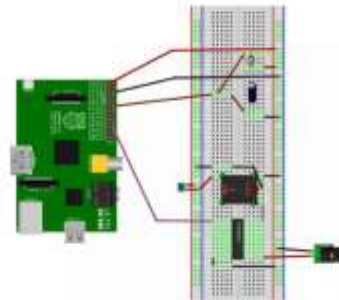
Multiple nodes:

- Installed in different locations

👉 Simple idea:

📌 “Build hardware system physically”

Step 9: Device & Component Integration



✓ Step 10: Application Development

Build:

- Cloud dashboard/web app

Features:

- Show graphs (temperature, humidity, etc.)
- Data visualization
- Predictions/analysis

- Remote control (change interval)

Simple idea:

“User sees and controls system”

Complete Flow (VERY IMPORTANT)

👉 End Node:

- Sensors → Raspberry Pi → Send data

👉 Cloud:

- Store → Analyze → Predict

👉 User:

- View data on dashboard

✓ 1. What does “Aggregated Data” mean?

👉 Aggregation = Combining data from multiple sources into one meaningful dataset

📌 In your weather system:

- Many nodes send data:
 - Node 1 → 25°C
 - Node 2 → 27°C
 - Node 3 → 26°C

👉 Aggregation means:

- Combine all this data together

✓ Example:

- Average temperature = $(25 + 27 + 26) / 3 = 26^{\circ}\text{C}$
- Min / Max values
- Area-wise summary

👉 So:

📌 Aggregation = “Collect + Combine + Summarize data”

✓ 2. What does “Analyzed Data” mean?

👉 After aggregation, the system studies the data to find useful information

✓ Examples:

- Detect trends (temperature increasing?)
- Predict weather (rain, heatwave)
- Detect abnormal values (sensor error)

👉 So:

📌 Analysis = “Understand the data and make decisions”



✓ 3. Role of Centralized Controller (VERY IMPORTANT)

👉 This is the "brain" of the system (in cloud)

🧠 What does it do?

◆ 1. Collects data

- Receives data from all nodes
-

◆ 2. Uses aggregated + analyzed data

- Uses results from cloud processing
-

◆ 3. Makes decisions

👉 Example decisions:

- Change sensing interval
 - Detect faulty node
 - Optimize system
-

◆ 4. Sends commands back to nodes

👉 Example:

- "Increase interval to 30 sec"
 - "Restart sensor"
 - "Stop sending data temporarily"
-

✓ 1. What type of decisions does the Centralized Controller make?

👉 Based on analyzed data, it decides:

- **Sampling rate decisions**
 - Increase/decrease data collection interval
 - **Fault detection**
 - Identify faulty sensors/nodes
 - **Data optimization**
 - Reduce unnecessary data transmission
 - **Environmental decisions**
 - Detect abnormal conditions (e.g., sudden temperature rise)
-

✓ 2. What type of commands are sent back?

👉 Controller sends control instructions like:

- Change sensing interval
 - 👉 (e.g., 15 sec → 60 sec)
 - Start / Stop data transmission
 - Reset / restart node
 - Calibration command
 - 👉 (adjust sensor accuracy)
 - Firmware/config updates
-

✓ 3. What type of IoT nodes are used?

👉 In weather monitoring system:

✓ **Sensor Nodes (End Nodes):**

- Temperature sensor node
- Humidity sensor node
- Pressure sensor node
- Light sensor node

✓ **Each node includes:**

- Sensors
 - Microcontroller / Raspberry Pi
 - Communication module (WiFi)
-